

Scheduling in Distributed Edge Serving for LLM’s

Problem Statement

Large language models (LLMs) are becoming increasingly capable, and tools built on top of them such as AI coding assistants, autonomous agents like Open Claw are beginning to demand more compute at inference time. As these systems grow in capability, the GPU requirements to serve them locally will continue to rise. High-end GPUs such as the NVIDIA H100 or A100, which are necessary to run large models efficiently, retail at \$25,000 to \$30,000 well beyond the reach of individual developers, researchers, and small teams.

At the same time, there exists a large and growing population of consumers who own capable but modest hardware laptops, desktops, and workstations with mid-range GPUs and who are willing to accept a trade-off: lower throughput and higher latency in exchange for the ability to run powerful models at all. This trade-off is currently underserved by existing solutions.

The core problem is therefore: *how can we enable distributed, collaborative inference across a heterogeneous pool of consumer-grade edge devices, without requiring each device to hold the entire model in memory?* The idea is to have GPU cluster at home or maybe distributed in home, office or maybe even in your pocket through phone. I got the idea from here [1] where they are trying to use cars as inference stations instead lets do normal edge devices.

Existing Solutions

Several systems have explored this space, each with notable strengths and limitations.

Petals

Petals [3] is an open-source framework developed by BigScience that allows users to run large language models collaboratively over the internet. Each participant hosts a subset of the model’s layers, and inference requests are routed through the available nodes using a Distributed Hash Table (DHT) — the same technology underlying BitTorrent. Petals demonstrated that distributed inference over the internet is technically feasible, achieving approximately 3–5 tokens per second for BLOOM-176B across geographically distributed nodes.

However, Petals has significant limitations. It supports only a fixed set of pre-approved models, requires users to run command-line tools to join the network, and is primarily designed for researchers rather than general consumers. It has not been updated to support the latest generation of models and is not production-ready.

Exo

Exo [?] is an open-source project by Exo Labs that enables users to cluster their own personal devices — MacBooks, Linux machines, iPhones, and Raspberry Pis — for local distributed inference. It uses automatic peer discovery via UDP broadcast, topology-aware model partitioning, and supports an OpenAI-compatible API. Exo has demonstrated impressive results, running Llama-3.1 405B across consumer Apple Silicon hardware.

However, Exo is designed primarily for devices on the same local network. It does not natively support internet-scale clustering across devices owned by different users in different locations, and as of late 2025 it

is not considered production-ready, with known gaps in security, fault tolerance, and operational tooling.

Distributed Llama

Distributed Llama [?] is a lightweight open-source project that connects home devices into a basic inference cluster via a command-line interface. It is functional but minimal — lacking automatic device discovery, intelligent planning, or a user-friendly interface. It is best described as a proof of concept rather than a deployable system.

Wavefy

Wavefy is an experimental decentralized inference framework that incorporates blockchain-based incentives for node participation. While the underlying idea of shortest-chain routing across distributed nodes is interesting, the addition of cryptocurrency mechanics adds unnecessary complexity for users who simply want to run a model.

Comparison with Existing Systems

Feature	Relay	Petals	Exo	Dist. Llama
Any HuggingFace model	✓	×	✓	Limited
Internet-scale clustering	✓	✓	×	×
User device registration	✓	CLI only	Local only	CLI only
Auto pipeline planning	✓	DHT-based	✓	×
OpenAI compatible API	✓	×	✓	×
Friendly UX	✓	×	Partial	×
Production ready	In progress	×	×	×

Our Idea

Overview

There are two things in inference as far as I know and read and this can be categorized into normal plane. Data plane where actual inference engines like vLLM, SGLang, llama.cpp, tensorrt-llm does prefill and decoder phase maintains KV caches and runs the model. The next is control plane, one example would be [Nvidia Dynamo](#) which tries to do load balancing, takes scheduling decisions etc.

I would love to operate on data plane and create an inference engine for this problem like Petals. But I don't have expertise, and also not sure what can I learn and implement in one month so will stick with only control plane, with data plane optimizations alone. I am thinking Distserve, and Turboquant approaches, but need to discuss with my teammates. These are my thoughts and below is my summary of the idea.

Let's say the user has 4 or 5 machines in house. He wants to setup a openclaw or a home agent which has access to multiple home control systems like AC etc. There are multiple persons in the home and they all want to access the AI agent like openclaw to operate on something. A traditional way is to serve on just one machine and expose that as API to OpenClaw for doing work. However, right now personal automation is increasing we may need availability. I believe in future my personal agent will be talking with your personal agent, and throughput is needed. Parallelism techniques like tensor and pipeline is introduced to serve a big model like 105B in your home consumer GPU, that is good but I am thinking what if my machine is at home network connected to each other and we need wireless. Previous techniques like exo wont work since they assume high speed data transfer as it is needed to do KV cache transferring. Techniques like petals is ok, but don't work on control plane logic. So that's why our idea Edge Serve(I have not decided on the name, open for suggestions :))

The idea is to have something like this, a user registers his device to a network by a dashboard saying what model he wants to choose with other configurable parameters like mentioned in the formula also the

format for high availability(etcd) v/s SWIM. High availability uses RAFT which prevents Single point of failure, while SWIM is designed if users ≥ 3 and user needs quick convergence. Also would be interested in comparison at end. One device is up for serving, we also get home cluster key once we sign up. The users can sign up with other devices on the same UI with the same cluster key. They join as followers and they get the model downloaded automatically from the main cluster. I think at this point both should be in the same network. Then next time when the request comes it goes to coordinator who is running the scheduling algorithm which uses its decision to route to the specific node based on a formula.

1 System Architecture

1.1 High-Level Design

HomeCluster follows a **centralized coordinator** architecture. All client requests — from systems such as OpenClaw, Home Assistant, or any OpenAI-compatible client — are directed to a single coordinator endpoint. The coordinator runs the scheduling algorithm, selects a worker, forwards the request, and streams the response back to the client. Workers are unaware of each other; they only report telemetry to the coordinator.

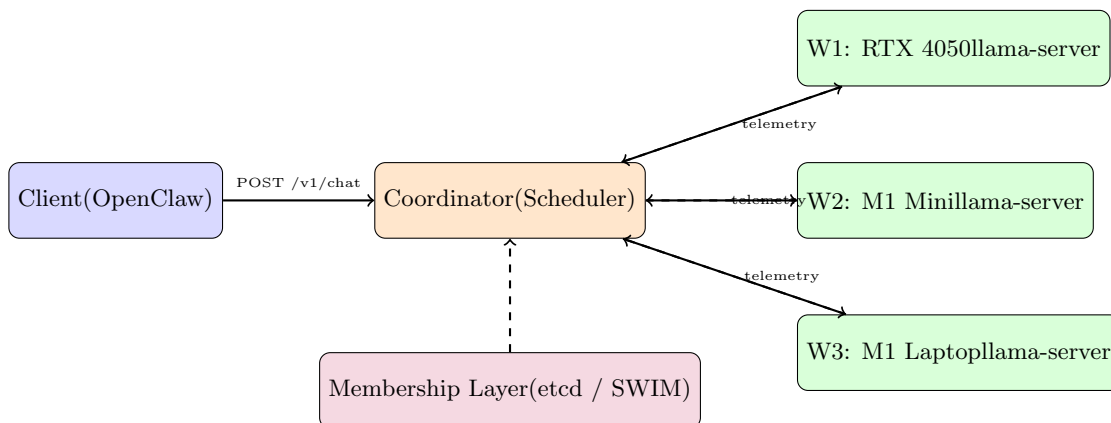


Figure 1: HomeCluster system architecture. All client traffic flows through the coordinator. Workers report telemetry every 200ms. The membership layer handles fault detection and leader election.

1.2 Parallelism Strategy: Data Parallelism

HomeCluster uses **data parallelism**: each worker holds a full copy of the model and processes requests independently. This is the correct choice for the target hardware and network environment for three reasons:

1. **No cross-device communication during inference.** Each request is handled entirely on one worker, making WiFi latency irrelevant during token generation.
2. **Linear capacity scaling.** Adding a device adds proportional throughput.
3. **Hardware fit.** A 7–8B model in Q4 quantization fits comfortably on both a 6 GB NVIDIA GPU and an M1 Mac with 16 GB unified memory.

This is contrasted with pipeline parallelism (used by exo, Petals, prima.cpp), where one model is split layer-by-layer across devices. Pipeline parallelism enables running models too large for any single device but introduces inter-device activation transfers whose latency is prohibitive over WiFi at consumer speeds.

1.3 Why Not Full Prefill-Decode Disaggregation (DistServe)

DistServe [6] and NVIDIA Dynamo [2] physically separate prefill (compute-bound) and decode (memory-bandwidth-bound) phases onto different machines. This requires transferring the KV cache between machines

after prefill completes.

For an 8B model with a 2000-token prompt:

$$\text{KV cache size} = L \times 2 \times H \times D \times T \times \text{bytes} = 32 \times 2 \times 32 \times 128 \times 2000 \times 2 \approx 500 \text{ MB} \quad (1)$$

At realistic home WiFi throughput of 50 MB/s, this transfer alone introduces ~ 10 seconds of latency before the first token is generated. Therefore, full prefill-decode disaggregation is not viable over WiFi. HomeCluster instead implements **phase-aware routing**: long-prompt requests are biased toward compute-strong workers (NVIDIA GPU) without any cross-device KV transfer.

2 Abstraction Layers

HomeCluster is designed around two core abstraction interfaces, making it membership-backend agnostic and inference-engine agnostic.

2.1 Membership Layer Interface

Listing 1: MembershipLayer interface

```
class MembershipLayer:
    def register(self, node_id, metadata): ...
    def deregister(self, node_id): ...
    def get_alive_members(self) -> list[Node]: ...
    def watch_membership(self, callback): ...
    def elect_leader(self) -> str: ...
    def is_leader(self) -> bool: ...
    def put(self, key, value): ...
    def get(self, key) -> value: ...
```

Two implementations are provided, selectable at onboarding:

Backend	Protocol	Recommended For	Fault Tolerance
etcd	Raft consensus	3+ node clusters	One node can die freely
SWIM gossip	Gossip + random probing	1-2 node clusters	Best-effort

Table 1: Membership layer backends.

2.2 Inference Engine Interface

Listing 2: InferenceEngine interface

```
class InferenceEngine:
    def generate(self, request) -> stream: ...
    def health(self) -> EngineHealth: ...
    def get_telemetry(self) -> Telemetry: ...
        # Returns: queue_depth, tokens_per_sec_ema,
        #           memory_pressure, cached_prefix_hashes
```

Two implementations are provided:

- **LlamaCppEngine** — Primary, fully tested. Supports NVIDIA CUDA and Apple Metal on the same cluster via the RPC backend. Exposes rich slot-level KV cache telemetry.
- **VllmEngine** — Secondary, best-effort. Linux + NVIDIA only. Exposes telemetry via Prometheus metrics endpoint.

2.3 NetworkLayer Interface

The network layer abstracts peer discovery and address resolution. All coordinator-to-worker communication uses addresses returned by this interface; the coordinator never hardcodes IPs or hostnames.

Listing 3: NetworkLayer interface

```
class NetworkLayer:
    def get_my_address(self) -> str: ...
    def get_peer_address(self, peer_id: str) -> str: ...
    def discover_peers(self) -> list[PeerInfo]: ...
    def is_peer_reachable(self, peer_id: str) -> bool: ...
```

Two implementations are provided:

- **TailscaleNetwork** — primary. Talks to the Tailscale LocalAPI via Unix domain socket (`/var/run/tailscale/tailscale.sock`). Returns stable `100.x.x.x` Tailscale IP addresses that remain constant across network changes, WiFi roaming, and DHCP reassignments. Enables cross-network clusters (devices on different home networks, cities, or countries) with no port forwarding or router configuration. Peer discovery reads the `/localapi/v0/status` endpoint.
- **LANNetwork** — fallback. Uses mDNS/DNS-SD (`_homecluster._tcp.local`) for zero-configuration LAN discovery. Resolves peers by hostname on the local subnet. Requires all devices to be on the same network. No external dependency.

2.4 Complete Abstraction Diagram

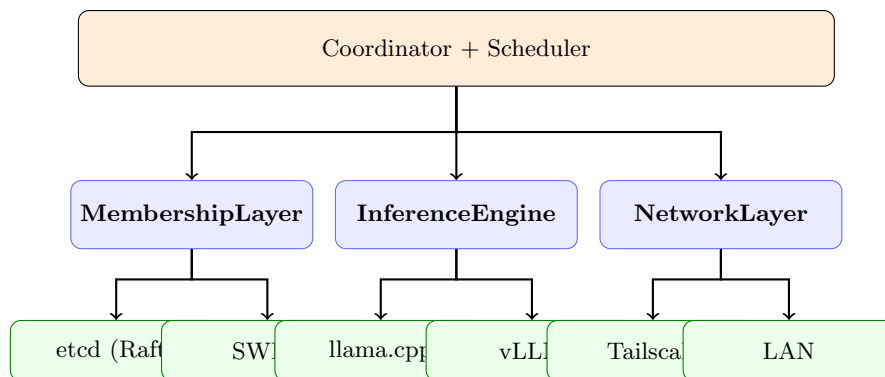


Figure 2: Three abstraction interfaces. The coordinator only imports the interfaces, never the implementations. All wiring is done at startup by a factory function driven by user configuration.

The three interfaces and their concrete implementations are:

Interface	Implementations	User selects at
<code>MembershipLayer</code>	etcd (Raft), SWIM (gossip)	First-run onboarding
<code>InferenceEngine</code>	llama.cpp, vLLM	Per-device onboarding
<code>NetworkLayer</code>	Tailscale, LAN (mDNS)	First-run onboarding

Table 2: The three abstraction interfaces and their concrete backends.

3 Control Plane

3.1 Raft vs SWIM

The two membership backends solve fundamentally different problems:

Property	SWIM (Gossip)	Raft (etcd)
Purpose	Failure detection, membership	Consensus on shared state
Consistency	Eventual (seconds of lag acceptable)	Strong (linearizable)
Requires quorum	No	Yes (majority must be alive)
Works with 1 node	Yes	Yes (single-node degraded mode)
Works with 2 nodes	Yes	No (loses quorum if one dies)
Works with 3 nodes	Yes	Yes (one can die freely)
Split-brain safe	No — multiple partitions may operate	Yes — only majority partition operates
Failure detection speed	3–5 seconds	~lease TTL (5–10 seconds)
Operational overhead	Zero (embedded library)	Low (embedded binary, auto-started)

Table 3: SWIM gossip vs Raft consensus comparison.

Key distinction on split-brain safety. With a 3-node cluster that partitions into $\{W_1\}$ and $\{W_2, W_3\}$:

- **SWIM:** Both partitions continue operating independently. W_1 believes it is the cluster; W_2, W_3 believe they are the cluster. Two simultaneous “truths” exist.
- **Raft:** W_1 cannot form a majority ($1/3 < 0.5$) and steps down. W_2, W_3 form a majority ($2/3 > 0.5$) and continue. Exactly one active partition — guaranteed by mathematics.

Two majorities cannot simultaneously exist in Raft because you cannot partition an odd-sized set into two halves both exceeding 50%. The co-ordinator or leader over here, will use **gRPC** to call other nodes.

4 Network Layer

HomeCluster abstracts all peer discovery and address resolution behind a **NetworkLayer** interface. The coordinator and scheduler never reference an IP address or hostname directly; they call only this interface.

Listing 4: NetworkLayer interface

```
class NetworkLayer:
    def get_my_address(self) -> str: ...
    def get_peer_address(self, peer_id: str) -> str: ...
    def discover_peers(self) -> list[PeerInfo]: ...
    def is_peer_reachable(self, peer_id: str) -> bool: ...
```

Two implementations are provided, selectable at onboarding:

- **TailscaleNetwork** — primary. Communicates with the local Tailscale daemon via its Unix domain socket LocalAPI. Returns stable `100.x.x.x` Tailscale IP addresses that remain constant across WiFi roaming, DHCP reassignment, and network changes. Enables clusters spanning different home networks, cities, or countries with no port forwarding or router configuration required.
- **LANNetwork** — fallback. Uses mDNS/DNS-SD (`_homecluster.tcp.local`) for zero-configuration discovery within a single subnet. No external dependency; requires all devices to be on the same local network.

4.1 The NAT Problem

Home devices sit behind Network Address Translation (NAT) routers and hold private IP addresses (e.g. 192.168.1.x) that are not routable from outside their subnet. Two devices on different home networks cannot initiate direct TCP connections because each router drops unsolicited inbound packets. Tailscale solves this by building a WireGuard-based encrypted mesh where each enrolled device receives a stable 100.x.x.x address routable to any other device on the same tailnet, regardless of physical network location. Tailscale attempts direct peer-to-peer hole-punching; if that fails it falls back to relaying through its DERP servers. All traffic is end-to-end encrypted.

4.2 TailscaleNetwork Implementation

Listing 5: TailscaleNetwork implementation via LocalAPI

```
import httpx

class TailscaleNetwork(NetworkLayer):

    def _localapi(self, path: str) -> dict:
        transport = httpx.HTTPTransport(
            uds="/var/run/tailscale/tailscaled.sock"
        )
        client = httpx.Client(
            transport=transport,
            base_url="http://local-tailscaled.sock"
        )
        return client.get(path).json()

    def get_my_address(self) -> str:
        status = self._localapi("/localapi/v0/status")
        return status["Self"]["TailscaleIPs"][0]

    def get_peer_address(self, peer_id: str) -> str:
        status = self._localapi("/localapi/v0/status")
        return status["Peer"][peer_id]["TailscaleIPs"][0]

    def discover_peers(self) -> list[PeerInfo]:
        status = self._localapi("/localapi/v0/status")
        return [
            PeerInfo(
                id = peer_id,
                address = info["TailscaleIPs"][0],
                online = info["Online"],
                name = info.get("HostName", peer_id)
            )
            for peer_id, info in status["Peer"].items()
        ]

    def is_peer_reachable(self, peer_id: str) -> bool:
        status = self._localapi("/localapi/v0/status")
        return status["Peer"].get(peer_id, {}).get("Online", False)
```

4.3 OpenClaw Integration

HomeCluster serves as the LLM backend for OpenClaw or any OpenAI-compatible client. OpenClaw natively supports Tailscale Serve (tailnet-only access) and Tailscale Funnel (public HTTPS). Since HomeCluster runs on the same tailnet, the integration requires a single configuration change in OpenClaw pointing to the coordinator's Tailscale IP:

Listing 6: OpenClaw LLM provider configuration

```
{
  "ai": {
    "provider": "openai-compatible",
    "baseUrl": "http://100.64.0.1:8080/v1",
    "apiKey": "homecluster",
    "model": "llama-3.1-8b"
  }
}
```

OpenClaw treats HomeCluster as a standard OpenAI endpoint. The scheduling, routing, fault-tolerance, and multi-device orchestration are entirely transparent to OpenClaw. Future network backends (e.g., Cloudflare Tunnel for public access without Tailscale, Headscale for self-hosted coordination) can be added as new `NetworkLayer` implementations without modifying any other component.

5 Device Onboarding and Trust Establishment

5.1 Design Goals

The onboarding system must: (1) require no technical expertise from the user, (2) establish cryptographically strong mutual trust between devices, (3) produce a permanent device identity that survives restarts and IP changes, and (4) require no central certificate authority or external trust infrastructure.

5.2 Persistent Trust Store

Each device maintains a local SQLite database as its trust store. This is the only persistent state required for the pairing system.

Listing 7: Trust store schema

```
-- Generated once at first run; never changes per device
CREATE TABLE my_identity (
  device_id TEXT PRIMARY KEY, -- SHA-256(public_key)
  public_key TEXT NOT NULL, -- Ed25519 public key (base64)
  private_key TEXT NOT NULL -- Ed25519 private key (encrypted at rest)
);

-- One row per trusted peer; written at pair time, read at every reconnect
CREATE TABLE trusted_peers (
  device_id TEXT PRIMARY KEY, -- SHA-256(peer_public_key)
  public_key TEXT NOT NULL, -- Ed25519 public key (base64)
  nickname TEXT, -- "M1 Mac Mini", "Gaming Laptop"
  tailscale_ip TEXT, -- 100.x.x.x, updated on reconnect
  added_at DATETIME,
  last_seen DATETIME
);
```

Once a peer is written to `trusted_peers`, the two devices recognise each other permanently across restarts, network changes, and Tailscale IP changes. No re-pairing is required unless a device is explicitly removed or reinstalled.

5.3 Pair-Code Flow

1. **Code generation.** The coordinator generates a cryptographically random 9-digit code displayed as XXX-XXX-XXX. A QR code encoding `homellm://pair?code=...&did=...&addr=...` is shown alongside. The code is single-use and expires after five minutes.

2. **Code entry.** The joining device scans the QR code or enters the numeric code manually. It extracts the coordinator’s Tailscale IP and initiates a connection.
3. **SPAKE2+ key exchange.** Both devices run the SPAKE2+ protocol (RFC 9383), a Password-Authenticated Key Exchange (PAKE), using the 9-digit code as the shared password. SPAKE2+ provides a formally proven guarantee: a passive eavesdropper who observes the full exchange cannot learn the code or derive the resulting session key. An active attacker gets exactly one online guess before the session is invalidated.
4. **Identity exchange.** Under the SPAKE2+-derived session key, both devices exchange their long-term **Ed25519** public keys and metadata (nickname, Tailscale IP). The device ID is **SHA-256(public_key)**, following the Synthing identity model. Both devices write the peer to their local **trusted_peers** table. The pair code is immediately invalidated.
5. **Noise_IK for all future RPCs.** All subsequent communication uses the Noise_IK handshake. Because both sides already hold each other’s public keys in the trust store, session establishment requires one round trip with no certificate negotiation.
6. **Cluster registration.** The new device registers in the membership layer, starts its **llama-server** subprocess, and begins publishing telemetry to the coordinator every 200 ms.

5.4 Reconnection After Restart

No re-pairing is needed after a restart. On startup each device reads **trusted_peers** from SQLite, connects to known peers via their stored Tailscale IPs, and re-authenticates via Noise_IK in one round trip with no user input. If a peer’s Tailscale IP has changed, **TailscaleNetwork** resolves the current address via the LocalAPI using the device ID and updates the trust store. Identity is tied to the Ed25519 public key, not the IP address.

Re-pairing is required only when a device is explicitly removed from the cluster, a device is reinstalled and loses its private key, or an administrator revokes a device it suspects is compromised.

6 Scheduling Algorithm

6.1 Telemetry Publishing

Each worker publishes the following telemetry to the coordinator every 200 ms:

- q_w — current queue depth (tokens waiting)
- $s_w(b)$ — exponential moving average of tokens/second for prompt-length bucket b
- m_w — GPU/unified memory pressure $\in [0, 1]$
- j_w — link RTT jitter EMA (milliseconds), measured from heartbeat timings
- θ_w — thermal flag $\in \{0, 1\}$, derived from comparing current GPU clock to base clock
- $\mathcal{H}_w$ — set of cached 16-token prefix chunk hashes currently held in KV cache

6.2 Prefix Hashing

To enable KV-cache-aware routing without transmitting full prompts, each prompt is divided into fixed chunks of 16 tokens. Each chunk is hashed:

$$h_i = \text{hash}(\text{chunk}_i), \quad i = 0, 1, \dots, \left\lfloor \frac{T}{16} \right\rfloor \quad (2)$$

The prefix overlap fraction for worker w given prompt hash sequence $[h_0, h_1, \dots, h_n]$ is:

$$\text{overlap}(w) = \frac{\arg \min_k \{h_k \notin \mathcal{H}_w\}}{n + 1} \quad (3)$$

where $\arg \min_k \{h_k \notin \mathcal{H}_w\}$ returns the index of the first chunk not present in worker w 's cache, which equals the count of contiguous cached chunks from the start of the prompt; dividing by $n + 1$ normalises this to a fraction in $[0, 1]$. For example, given three chunks with hashes $[h_0, h_1, h_2]$ and a worker w whose cache contains $\{h_0, h_1\}$ but not h_2 , the first miss occurs at $k = 2$, giving $\text{overlap}(w) = 2/3 \approx 0.67$. A worker with an empty cache gives $k = 0$ and $\text{overlap} = 0$; a worker missing h_1 but holding h_2 still gives $k = 1$ and $\text{overlap} = 1/3$, since contiguity from the start is broken at h_1 and the cached h_2 is unusable.

6.3 Cost Function

For each incoming request r with prompt length T tokens, the scheduler computes a cost for each candidate worker w :

$$\text{cost}(w, r) = \alpha \cdot \frac{q_w}{s_w(b)} + \beta \cdot (1 - \text{overlap}(w, r)) + \gamma \cdot m_w + \delta \cdot \frac{j_w}{j_{\max}} + \varepsilon \cdot \theta_w \quad (4)$$

where the terms are:

Term	Weight	Meaning
$\alpha \cdot \frac{q_w}{s_w(b)}$	α	Estimated queue wait time at worker w given current load and device speed in prompt-length bucket b
$\beta \cdot (1 - \text{overlap}(w, r))$	β	Penalty for KV cache miss — proportion of prompt that must be re-prefilled
$\gamma \cdot m_w$	γ	Memory pressure penalty — avoid workers approaching OOM
$\delta \cdot \frac{j_w}{j_{\max}}$	δ	Wireless jitter penalty — penalise workers on flaky WiFi links
$\varepsilon \cdot \theta_w$	ε	Thermal penalty — penalise workers currently throttling below base clock

Table 4: Cost function terms. Terms marked in bold are the novel edge-specific contributions absent from prior datacenter schedulers.

The scheduler selects:

$$w^* = \arg \min_{w \in \mathcal{W}_{\text{alive}}} \text{cost}(w, r) \quad (5)$$

6.4 Phase-Aware Routing

For long-prompt requests ($T > K$, default $K = 1024$), a phase bias term is added to favour compute-strong workers for the prefill-dominant phase:

$$\text{cost}_{\text{phase}}(w, r) = \text{cost}(w, r) + \phi \cdot \mathbf{1}[T > K] \cdot (1 - c_w) \quad (6)$$

The indicator $\mathbf{1}[T > K]$ acts as a binary switch: it evaluates to 1 when the prompt length T exceeds the threshold $K = 1024$ tokens (prefill-dominant), and 0 otherwise, in which case the phase term contributes nothing and routing falls back to the base cost function. The term $(1 - c_w)$ captures the compute weakness of worker w : a compute-strong worker such as the RTX 4050 ($c_w = 0.9$) incurs a small penalty of 0.1, while a compute-weaker worker such as the M1 Mini ($c_w = 0.4$) incurs a larger penalty of 0.6, steering long-prompt requests toward the higher-TFLOPS device without requiring any cross-device KV cache transfer.

For example, given $\phi = 1.0$, $c_{W_1} = 0.9$ (RTX 4050), and $c_{W_2} = 0.4$ (M1 Mini), a short prompt ($T = 200 < K$) leaves both workers unaffected ($\mathbf{1}[T > K] = 0$), while a long prompt ($T = 3000 > K$) adds a phase penalty of 0.1 to W_1 and 0.6 to W_2 , making W_1 the preferred worker for the prefill-heavy request.

6.5 SLO-Aware Admission Control

If the estimated time-to-first-token for a new request exceeds a configurable SLO threshold τ_{TTFT} :

$$\widehat{\text{TTFT}}(w^*, r) = \frac{q_{w^*}}{s_{w^*}(b)} + \frac{T}{s_{w^*}^{\text{prefill}}} > \tau_{\text{TTFT}} \quad (7)$$

To illustrate, consider worker W_1 (RTX 4050) with the following state at the moment a new request arrives:

Variable	Value	Meaning
q_{w^*}	2000 tokens	tokens currently queued at W_1
$s_{w^*}(b)$	100 tok/s	speed at this prompt-length bucket
T	1000 tokens	incoming request prompt length
$s_{w^*}^{\text{prefill}}$	500 tok/s	prefill speed
τ_{TTFT}	10 seconds	SLO threshold

Table 5: Worker state for SLO admission control example.

Substituting into Equation (7):

$$\widehat{\text{TTFT}}(W_1, r) = \underbrace{\frac{2000}{100}}_{\text{queue wait}=20\text{s}} + \underbrace{\frac{1000}{500}}_{\text{prefill}=2\text{s}} = 22 \text{ s} > \tau_{\text{TTFT}} = 10 \text{ s}$$

Since 22 seconds exceeds the SLO threshold, the request is queued at the coordinator or shed with a 503 response rather than forwarded to W_1 . Conversely, if W_1 's queue were light ($q_{w^*} = 200$ tokens), the estimate would be $2 + 2 = 4$ seconds $< \tau_{\text{TTFT}}$, and the request would be accepted and forwarded immediately.

Run	λ configuration	Effective policy
1	all $\lambda = 1$	Full system (all terms active)
2	$\lambda_\beta = 0$, rest = 1	No KV cache awareness
3	$\lambda_\delta = 0$, rest = 1	No jitter awareness
4	$\lambda_\varepsilon = 0$, rest = 1	No thermal awareness
5	$\lambda_\alpha = 1$, rest = 0	Least-loaded baseline
6	all $\lambda = 0$	Round-robin baseline

Table 6: Ablation runs. Each run zeroes one term independently to isolate its contribution to goodput and TTFT. Runs 5 and 6 degenerate to standard baselines used for comparison.

6.6 Ablation Design

Each coefficient can be zeroed independently to measure its contribution:

$$\text{cost}_{\text{ablation}}(w, r; \boldsymbol{\lambda}) = \lambda_\alpha \alpha \cdot \frac{q_w}{s_w(b)} + \lambda_\beta \beta \cdot (1 - \text{overlap}) + \lambda_\gamma \gamma \cdot m_w + \lambda_\delta \delta \cdot \frac{j_w}{j_{\text{max}}} + \lambda_\varepsilon \varepsilon \cdot \theta_w \quad (8)$$

where $\lambda_x \in \{0, 1\}$ is a binary ablation switch. Setting all $\lambda = 0$ except λ_α degenerates to least-loaded routing. Setting all $\lambda = 0$ degenerates to round-robin.

6.7 Telemetry Specification

Each worker publishes a **Telemetry** object to the coordinator every 200 ms. Signals are obtained either directly from the engine’s metrics endpoint or computed by the worker shim wrapper.

Signal	llama.cpp source	vLLM source
q_w queue depth	/metrics: llamacpp_requests_processing	/metrics: vllm:num_requests_waiting
m_w memory pressure	/metrics: llamacpp_kv_cache_usage_ratio	/metrics: vllm:gpu_cache_usage_perc
s_w^{prefill} prefill speed	Shim timestamps first token vs request sent	/metrics: vllm:time_to_first_token_seconds

Table 7: Signals obtained directly from engine endpoints.

Signal	How shim computes it (same for both engines)
$s_w(b)$ speed per bucket	Shim wraps <code>generate()</code> , records tok/s per completed request, and maintains an EMA per prompt-length bucket.
$\mathcal{H}_w$ prefix hashes	Shim hashes every processed prompt into 16-token chunks, tracks the active set, and evicts on memory pressure.
j_w link jitter	Shim sends a heartbeat ping to the coordinator every 200 ms, records RTT, and computes standard deviation as a jitter EMA.
θ_w thermal flag	NVIDIA: <code>nvidia-smi --query-gpu=clocks.gr,clocks.max.gr</code> ; throttling if current < 90% of max clock. Apple: <code>powermetrics --samplers gpu_power</code> .

Table 8: Signals computed by the worker shim, not exposed natively by any inference engine. The shim absorbs all engine-specific differences so the coordinator always receives an identical **Telemetry** object regardless of which engine is running on the device.

6.8 Scheduling Algorithm

On arrival of each request r , the coordinator executes the following procedure:

1. Receive request r with prompt length T tokens.
2. Retrieve the latest telemetry snapshot from all alive workers $w \in \mathcal{W}_{\text{alive}}$ (published every 200 ms via the membership layer).
3. For each worker w , compute prefix overlap:
 - Chunk the prompt into 16-token blocks and hash each block.
 - Walk the hash list from the start; stop at the first hash not found in $\mathcal{H}_w$.
 - Compute $\text{overlap}(w, r)$ using Equation (3).
4. For each worker w , compute the base cost using Equation (4):
 - Substitute q_w , $s_w(b)$, $\text{overlap}(w, r)$, m_w , j_w , θ_w from telemetry.
 - Select the speed bucket b corresponding to prompt length T .
5. If $T > K$ (default $K = 1024$ tokens), apply the phase-aware bias using Equation (6):
 - Add $\phi \cdot (1 - c_w)$ to each worker’s cost.
 - This steers long-prompt requests toward compute-strong workers.

6. Select the best worker:

$$w^* = \arg \min_{w \in \mathcal{W}_{\text{alive}}} \text{cost}_{\text{phase}}(w, r)$$

7. Check SLO admission using Equation (7):

- Estimate $\widehat{\text{TTF}}(w^*, r) = \frac{q_{w^*}}{s_{w^*}(b)} + \frac{T}{s_{w^*}^{\text{prefill}}}$.
- If $\widehat{\text{TTF}} > \tau_{\text{TTF}}$: queue the request at the coordinator or shed with a 503 response.
- If $\widehat{\text{TTF}} \leq \tau_{\text{TTF}}$: forward the request to w^* .

8. Stream tokens from w^* back to the client via the coordinator.

9. On request completion, the worker shim updates its EMA trackers (speed per bucket, prefill speed) and refreshes $\mathcal{H}_w$ with the newly cached prefix hashes.

6.9 Scheduling Flowchart

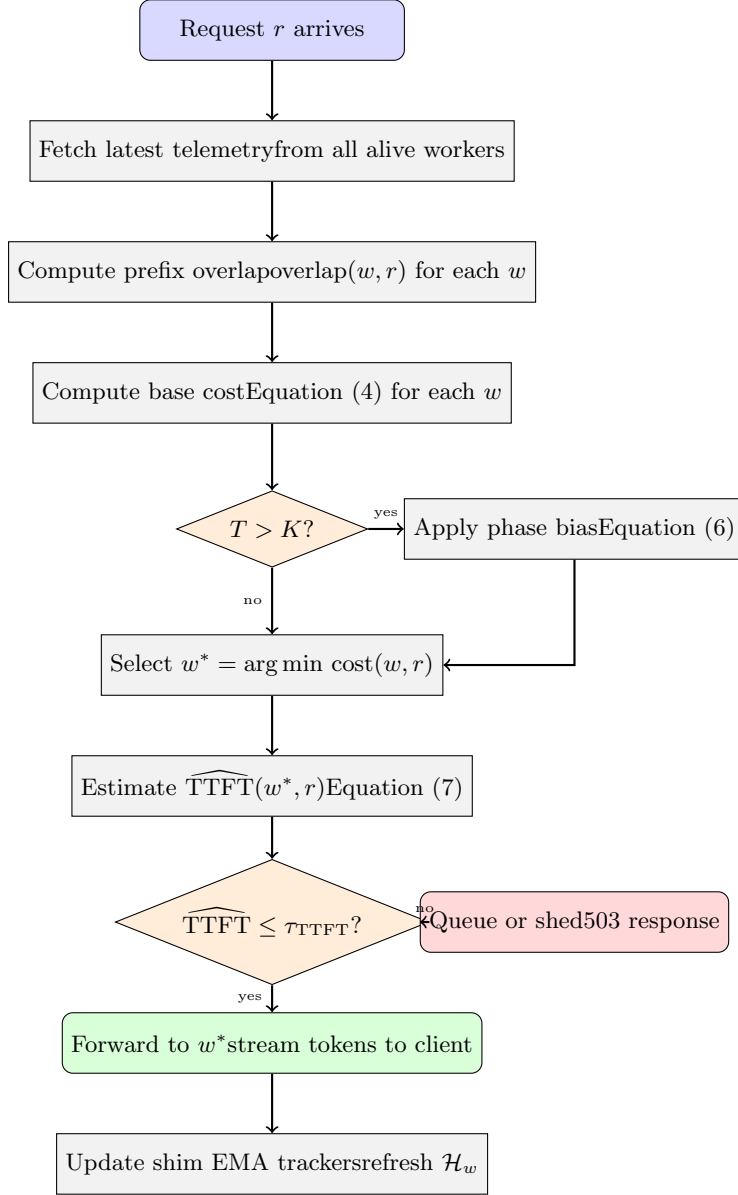


Figure 3: Scheduling flowchart. Each incoming request passes through prefix hashing, cost computation, optional phase bias, worker selection, and SLO admission control before being forwarded to the chosen worker.

7 Packaging and CLI

HomeCluster is distributed as a single Python package installable via `pip`. The daemon manages all subprocesses (llama-server, etc) and serves both the OpenAI-compatible API (`:8080/v1`) and the web dashboard (`:3000`) from a single FastAPI application. The CLI and the dashboard are two clients of the same daemon REST API; neither has special code paths.

7.1 CLI Command Reference

Listing 8: HomeCluster CLI commands

```
#      Installation

pip install homecluster

#      First device: create a new cluster

homecluster init
# Generates Ed25519 identity, prompts for membership backend
# (etcd / SWIM) and network backend (Tailscale / LAN),
# writes config to ~/.homecluster/config.json

homecluster start
# Starts daemon, opens dashboard at http://localhost:3000

#      Model management

homecluster pull llama-3.1-8b
# Downloads GGUF, restarts llama-server with new model

homecluster models list
# Shows which models are loaded on which cluster devices

#      Second device: join an existing cluster

homecluster join --code 482-719-305
# Connects to coordinator, runs SPAKE2+, exchanges Ed25519 keys,
# writes peer to trust store, starts worker, begins telemetry

#      Operations

homecluster status      # shows all devices, health, queue depth
homecluster logs        # tails daemon logs
homecluster stop        # graceful shutdown

#      From the second run onwards, only one command is needed

homecluster start
# Reconnects to all trusted peers automatically; no re-pairing
```

7.2 Complete Setup Walkthrough

Listing 9: End-to-end setup from zero to running inference

```
#      Device A (NVIDIA laptop)

$ pip install homecluster
$ homecluster init
-> Generating Ed25519 identity...           done
-> Network:      [1] LAN   [2] Tailscale    choice: 2
-> Tailscale IP detected: 100.64.0.1
-> Membership:  [1] SWIM  [2] etcd         choice: 1
-> Config written to ~/.homecluster/config.json
```

```

$ homecluster pull llama-3.1-8b
-> Downloading llama-3.1-8b.Q4_K_M.gguf (4.3 GB) ... done
-> llama-server started on :8081

$ homecluster start
-> Coordinator started on :8080
-> Dashboard at http://localhost:3000
-> Opening browser...

# Device B (M1 Mac Mini)

$ pip install homecluster
$ homecluster join --code 482-719-305
-> Connecting to 100.64.0.1 ...
-> SPAKE2+ key exchange ... done
-> Ed25519 identity exchange ... done
-> Peer stored: NVIDIA Laptop (abc123...)
-> Pulling llama-3.1-8b ... done
-> Worker started on :8081
-> Telemetry flowing to coordinator
-> Device B visible in Device A dashboard

# OpenClaw now points to http://100.64.0.1:8080/v1
# Scheduler routes across both devices automatically

# Every subsequent start (both devices)

$ homecluster start
-> Reconnecting to trusted peers ... done
-> No re-pairing required

```

8 Evaluation Plan

8.1 Metrics

The following metrics are reported, consistent with DistServe [6], Dynamo [2], and Mooncake [8]:

- **TTFT** — Time to First Token (ms). Primary latency metric.
- **TPOT** — Time Per Output Token (ms). Decode throughput per request.
- **ITL** — Inter-Token Latency (ms). Consistency of decode stream.
- **E2E Latency** — Total request completion time (ms).
- **Throughput** — Requests/second and tokens/second at the cluster level.
- **Goodput** — Fraction of requests meeting both TTFT and TPOT SLOs at a given arrival rate. This is the *headline metric*.
- **Fairness** — Jain’s fairness index on per-request normalised slowdown.
- **Cache Hit Rate** — Fraction of prompt tokens served from worker-resident prefix KV cache.
- **P95/P99 Tail Latency** — Tail behaviour under load.

8.2 Baselines

Baseline	Description	Comparable?
Round-Robin	Distribute requests cyclically, no state	Direct
Least-Queue	Route to worker with fewest queued tokens	Direct
KV-Aware (Dynamo-lite)	Longest prefix hash match, tie-break by queue	Direct
Full algorithm (ours)	Complete cost function with all terms	—
Ablations ($-\delta$, $-\varepsilon$, $-\beta$, $-\phi$)	Each term zeroed independently	Direct
NVIDIA Dynamo [2]	Datacenter disaggregated serving	Not apple-to-apple
Petals [3]	DHT collaborative inference	Not apple-to-apple
exo	Pipeline parallel consumer cluster	Partial

Table 9: Baselines. Direct baselines run on the same hardware under the same conditions. Not-apple-to-apple baselines are cited for context and positioning only.

8.3 Workloads

Workload	Characteristics	What it stresses
ShareGPT V3	Mixed short/medium chat, real user distribution	General routing, prefix reuse
LongBench	Long-context prompts (4k–16k tokens)	Phase-aware routing, TTFT
Azure LLM Inference 2024 [16]	Production arrival patterns, bursty	Admission control, tail latency
Mooncake conversation subset [8]	Up to 75% prefix hit rate in production	Cache-aware routing wins
Synthetic bursty	8 concurrent chats + 3 reasoning requests	Mixed workload, scheduler balance

Table 10: Evaluation workloads.

8.4 Experimental Conditions

Each workload is run under two conditions:

- **Wired LAN** — Gigabit ethernet, stable RTT < 1 ms. Establishes performance upper bound.
- **WiFi** — 5 GHz Wi-Fi 6, isolated SSID, realistic RTT 5–35 ms with jitter. Primary condition for this work.

Arrival rate is swept in open-loop Poisson mode for goodput curves. Closed-loop fixed concurrency is used for peak-throughput measurements.

8.5 Figures

1. **Latency CDF** (log-x axis) — TTFT distribution across all policies on ShareGPT.
2. **Goodput vs Arrival Rate** — Fraction of requests meeting SLO; headline comparison figure.
3. **Throughput-Latency Pareto** — P50 TTFT vs req/s, one curve per policy.
4. **Wired - Wireless connection** — Just to understand how network impacts performance.

5. **Per-Device Utilisation Timeline** — 60-second window, one subplot per policy, showing load balance.
6. **Ablation Bar Chart** — Goodput: Full vs $-\beta$ vs $-\delta$ vs $-\varepsilon$ vs $-\phi$.
7. **Cache Hit Rate vs Policy** — On Mooncake subset; shows KV-aware routing contribution.
8. **TTFT vs Prompt Length** — Phase-aware routing on LongBench; shows compute-bias win.
9. **Scheduler Decision Scatter** — Incoming requests coloured by prompt length, y-axis = chosen worker; makes routing logic visually legible.

9 Key Design Decisions Summary

Decision	Choice	Rationale
Parallelism	Data parallelism	No cross-device communication during inference; WiFi compatible
Inference engine	llama.cpp (primary), vLLM (secondary)	Only engine supporting NVIDIA + Apple Silicon in same cluster
Coordinator topology	Centralised	Global state visibility; simple debuggability; < 30 ms overhead negligible vs TTFT
Membership backend	etcd (3+ nodes) / SWIM (1–2 nodes), user-selectable	etcd for Raft correctness; SWIM for simplicity at small scale
KV transfer	None	WiFi makes cross-device KV transfer prohibitively slow
Phase disaggregation	Phase-aware routing only	Captures placement benefit without KV transfer engineering
Abstraction	MembershipLayer + InferenceEngine interfaces	Backend-agnostic; testable; extensible
Pair-code auth	SPAKE2+ + Ed25519 + Noise_IK	Formally proven PAKE; zero PKI; follows Matter/Synthing pattern

Table 11: Summary of key design decisions.

Limitations and Future Work

Relay in its current form has several known limitations:

- **Latency:** Internet-based inter-device communication is significantly slower than NVLink within a server. Throughput will be lower than single-machine inference.
- **Fault tolerance:** If a device drops out mid-inference, the current request fails. Future versions will implement redundant node assignment.
- **Security:** Intermediate activation tensors are passed between untrusted devices. Privacy-preserving inference techniques such as homomorphic encryption are a future research direction.
- **Dynamic models:** Relay currently requires static computational graphs with known tensor shapes at plan time.

Future work includes support for speculative decoding across pipeline stages, KV cache management across devices, and a web-based dashboard for cluster management and monitoring.

Summary

It seems a lot, but a lot of things are just library calls like llama.cpp and etcd implementation, the main thing is just scheduler and it is interesting to combine everything together.

Challenges

Listing out the challenges, I am not familiar about :

- **Inference Engine** : Building a inference engine and offloading layers, maintaining KV cache, seems interesting, but I have no expertise, need to learn a lot here. I ideally want turboquant also included in the Inference engine being used, but let's see.
- **Network Optimizations** : By design I know network would be the biggest bottle neck. Comparing normal Wired vs wireless I think TPS will be very less and would be a intersting observation to look into, but how optimizing maybe just keep it all in a single network
- **Consistency** : Most interesting part for me in P2P applications. Using etcd should solve it but bundling all these together would be great challenge. Lets see.
- **Formula** : It's a inital formula, I designed keeping few things in mind, but it can change later due to experiments.

Deliverables

- **Open-AI Compatible Serving Interface** : The system should be able to handle and serve on multiple frameworks while staying inference engine agnostic.
- **Ability to add devices** : The framework should be able to handle new devices and respective framework
- **CLI** : A CLI compatible with easy to run settings for the end user.
- **Planning algorithm** : System should be able to do optimal planning and scheduling for all devices. KV aware cache scheduling can be good. Need to work more on over here.
- **Telemetrys and comparison** Need to spend a considerable amount of time atleast one week gathering results. What about prometheus and grafana. Need to think.

10 Sample Mock Images

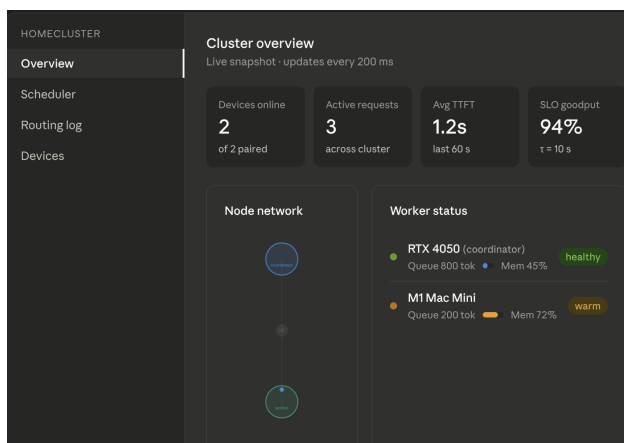


Figure 4: Co-Ordinator Dashboard

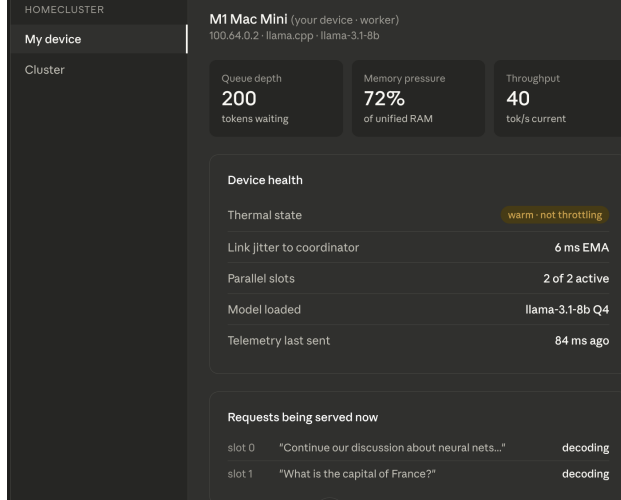


Figure 5: Follower Dashboard

11 Related Work

NVIDIA Dynamo [2] is the primary architectural reference. Its KV-aware routing cost function, disaggregated prefill/decode design, and SLA-based planner directly inform HomeCluster’s scheduler. However, Dynamo targets datacenter deployments with NVLink/InfiniBand and Linux-only backends; it is not applicable to consumer WiFi or Apple Silicon.

Petals [3] enables collaborative inference over DHT across thousands of volunteers. Its DHT-based membership, pipeline-parallel model sharding, and handling of unreliable peers are studied but not adopted: the DHT overhead and Byzantine-tolerance mechanisms are unnecessary for a trusted 2–5-node private LAN.

Prima.cpp [4] is the closest published match to this project’s hardware target, proposing piped-ring parallelism and an ILP-based layer placer for heterogeneous home clusters over WiFi. HomeCluster differs in using data parallelism rather than pipeline parallelism, enabling request-level KV-aware routing.

Helix [5] formulates heterogeneous GPU serving as a max-flow MILP problem, providing the theoretical foundation for heterogeneity-aware placement used in HomeCluster’s phase-aware routing term.

DistServe [6] and **Splitwise** [7] introduce goodput-under-SLO as the primary metric for disaggregated serving evaluation, adopted directly by HomeCluster.

Mooncake [8] demonstrates up to 75% prefix cache hit rates in production and provides the conversation trace used in evaluation.

Preble [9] introduces distributed prefix-aware scheduling with a hierarchical global + per-GPU scheduler. HomeCluster’s KV-aware term (β) is directly inspired by Preble’s E2 algorithm, extended with the novel wireless (δ) and thermal (ε) terms.

Llumnix [10] contributes KV-aware load balancing and live request migration patterns, and the S-S/M-M/L-L workload decomposition used in evaluation.

References

- [1] Tesla Cars as Inference. *Tesla Cars as Inference*. <https://www.datacenterdynamics.com/en/news/elon-musk-proposes-using-bored-tesla-cars-as-mobile-inferencing-fleet/>

- [2] NVIDIA Corporation. *NVIDIA Dynamo: A Low-Latency Distributed Inference Framework for Scaling Reasoning AI Models*. NVIDIA Technical Blog, 2025. <https://github.com/ai-dynamo/dynamo>
- [3] A. Borzunov et al. *Petals: Collaborative Inference and Fine-tuning of Large Models*. ACL 2023. arXiv:2209.01188
- [4] Li et al. *Prima.cpp: Fast 30–70B LLM Inference on Heterogeneous and Low-Resource Home Clusters*. ICLR 2026. arXiv:2504.08791
- [5] Mei et al. *Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow*. ASPLOS 2025. arXiv:2406.01566
- [6] Zhong et al. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI 2024. arXiv:2401.09670
- [7] Patel et al. *Splitwise: Efficient Generative LLM Inference Using Phase Splitting*. ISCA 2024. arXiv:2311.18677
- [8] Qin et al. *Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving*. FAST 2025. arXiv:2407.00079
- [9] Srivatsa et al. *Preble: Efficient Distributed Prompt Scheduling for LLM Serving*. ICLR 2025. arXiv:2407.00023
- [10] Sun et al. *Llumnix: Dynamic Scheduling for Large Language Model Serving*. OSDI 2024. arXiv:2406.03243
- [11] Jiang et al. *HexGen: Generative Inference of Large Language Model over Heterogeneous Environment*. ICML 2024. arXiv:2311.11514
- [12] *HexGen-2: Disaggregated Generative Inference of LLMs in Heterogeneous Environment*. arXiv:2502.07903
- [13] Griggs et al. *Mélange: Cost Efficient Large Language Model Serving by Exploiting GPU Heterogeneity*. arXiv:2404.14527
- [14] Miao et al. *SpotServe: Serving Generative Large Language Models on Preemptible Instances*. ASPLOS 2024. arXiv:2311.15566
- [15] Gradient Network. *Parallax: Efficient LLM Inference Service over Decentralized Environment*. arXiv:2509.26182
- [16] Microsoft Azure. *Azure LLM Inference Dataset 2024*. <https://github.com/Azure/AzurePublicDataset>
- [17] Das, Gupta, Motivala. *SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol*. DSN 2002.
- [18] *Lifeguard: Local Health Awareness for More Accurate Failure Detection*. arXiv:1707.00788
- [19] *RFC 9383: SPAKE2+, an Augmented Password-Authenticated Key Exchange (PAKE) Protocol*. IETF, 2023.
- [20] Zheng et al. *SGLang: Efficient Execution of Structured Language Model Programs*. arXiv:2312.07104
- [21] Wu et al. *LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism*. SOSP 2024. arXiv:2404.09526
- [22] OpenAI. *Chat Completions API Reference*. <https://platform.openai.com/docs/api-reference/chat>